

Exercício – Padrão Strategy (e-commerce)

Problema

Em um sistema de e-commerce, as regras de cálculo para frete seguem abaixo:

```
public class Pedido {

    private double peso;
    private String tipoFrete;

    public Pedido(double peso, String tipoFrete) {
        this.peso = peso;
        this.tipoFrete = tipoFrete;
    }

    public double calcularFrete() {
        if (tipoFrete.equalsIgnoreCase("SEDEX")) {
            return peso * 5 + 20;
        } else if (tipoFrete.equalsIgnoreCase("PAC")) {
            return peso * 2 + 10;
        } else if (tipoFrete.equalsIgnoreCase("RETIRADA")) {
            return 0;
        } else if (tipoFrete.equalsIgnoreCase("TRANSPORTADORA")) {
            return peso * 7 + 50;
        }

        throw new IllegalArgumentException("Tipo de frete inválido");
    }
}

public class Main {

    public static void main(String[] args) {
        Pedido pedido1 = new Pedido(10, "SEDEX");
        System.out.println("SEDEX: R$ " + pedido1.calcularFrete());

        Pedido pedido2 = new Pedido(10, "PAC");
        System.out.println("PAC: R$ " + pedido2.calcularFrete());

        Pedido pedido3 = new Pedido(10, "RETIRADA");
        System.out.println("Retirada: R$ " + pedido3.calcularFrete());

        Pedido pedido4 = new Pedido(10, "TRANSPORTADORA");
        System.out.println("Transportadora: R$ " + pedido4.calcularFrete());
    }
}
```

Pergunta: O que acontece com a classe Pedido toda vez que uma nova forma de frete precisa ser adicionada? Em outras palavras, qual o problema com esta solução?

Exercício Proposto

Objetivo

Implementar o padrão de projeto **Strategy** para permitir diferentes algoritmos de cálculo de frete em um sistema de e-commerce, sem alterar a classe principal do pedido.

Uma loja virtual precisa calcular o valor do frete de diferentes formas:

- **Sedex**: entrega rápida
- **PAC**: entrega econômica
- **Retirada na loja**: sem custo
- **Transportadora Expressa**: valor baseado no peso

Atualmente, o sistema possui vários if/else, dificultando manutenção e expansão. Você deve **refatorar** o sistema utilizando o padrão **Strategy**.

1. Criar a interface Strategy

A interface deve definir o método: `double calcularFrete(double peso);`

2. Criar as estratégias concretas

Implemente as estratégias de cálculo para Sedex, PAC, RetiradaNaLoja e TransportadoraExpressa.

3. Criar a classe Context

Implemente a classe Pedido, que deve:

- Receber uma estratégia de frete
- Permitir trocar a estratégia em tempo de execução
- Calcular o frete utilizando a estratégia atual

4. Criar um programa principal

No main:

- Criar um pedido com peso de 10kg
- Testar todas as estratégias
- Exibir o valor do frete para cada uma

5. Incluir uma regra nova: Frete Internacional

- Associar uma taxa de importação de 25.0 e multiplicar o peso da compra por 12
- Testar a nova estratégia

Questões discursivas

Ao final, responda as questões:

1. Qual problema o padrão Strategy resolve?
2. O que aconteceria se usássemos apenas if/else?
3. Como o sistema fica aberto para extensão?
4. O que acontece se cada país tiver uma regra de cálculo de frete diferente no caso de frete internacional?
5. Em quais outros cenários Strategy poderia ser utilizado?

Desafio extra

Imagine que o negócio expandiu e os donos agora querem incluir regras promocionais para os clientes. Como seria se regras promocionais pudessem ser aplicadas sobre uma estratégia existente, por exemplo:

- Frete grátis acima de R\$ 399

Responda:

1. Essas regras são novas estratégias de cálculo ou promoções?
2. Elas deveriam modificar Sedex, PAC, Internacional e demais regras de cálculo?
3. Como evitar alterar todas as classes já implementadas?